

Driver for Adafruit Neo Trinkey - SAMD21 USB Key

Samuele Lo Piccolo

August 15, 2026

Contents

1	Project data	1
2	Project description	1
2.1	Design and implementation	2
2.1.1	Init	2
2.1.2	Exit	2
2.1.3	Probe	2
2.1.4	Disconnect	2
2.1.5	Write	2
2.1.6	Read	3
3	Project outcomes	3
3.1	Concrete outcomes	3
3.2	Learning outcomes	3
3.3	Existing knowledge	3
3.4	Problems encountered	3
4	Honor Pledge	3

1 Project data

- Project supervisor: Vittorio Zaccaria
- The group that is delivering this project:

Last and first name	Person code	Email address
Lo Piccolo Samuele	10796320	samuele.lopiccolo@mail.polimi.it

- The Github repository can be found **here**

2 Project description

This projects allows to communicate with a serial interface exposed as a USB-CDC endpoint on the Adafruit Neo Trinkey. The aim of the project is to be able to send textual commands to the Circuit Python REPL and control the device's behaviour, mainly to change the LEDs' colours.

I did not implement the handling of multiple devices since I only have access to one Neo Trinkey and could not reliably validate the correctness of the solution

2.1 Design and implementation

My driver registers itself with the USB subsystem as a USB driver, providing only the required fields of **struct usb_driver**:

- The driver name
- The USB device table to be able to call the probe function
- The probe function that readies the driver and the device for communication
- The disconnect function that handles the disconnection of the device

The physical device is exposed as a character device in the `/dev/` directory, the only file operations implemented are read and write.

2.1.1 Init

The Init function registers my driver to USBcore so that when a match for the *Module Device Table* is found **Probe** will be called.

Init then goes on to:

- Request a character device region
- Initializing and registering with the kernel the character device associated with the module
- Creating a class of devices "aos_class"

2.1.2 Exit

The Exit function starts by unregistering from USBcore so that the disconnect function is called if a device is currently plugged in, then it deletes and unregisters the character device and finally it destroys the created class.

2.1.3 Probe

The Probe function is used to set the necessary CDC parameters for communication, it does so by sending two *USB Control Messages* to the device.

The first contains the **Line Coding** parameters which determine how the device and the host communicate.

The second control message instead sets up the CDC endpoint to accept bulk transfers, in particular I found that setting both DTR and RTS to 1 worked best.

If the device setup is successful I create a device file `/dev/AOSdev`

2.1.4 Disconnect

When Disconnect is called I destroy the device file `/dev/AOSdev` and empty a local buffer of characters read from the device, but not yet sent to userspace.

2.1.5 Write

In the Write function I make use of USB bulk transfers to send data to the device, since **usb_bulk_msg** only accepts kernel memory I must first create a local buffer and copy the userspace buffer to it. The device accepts up to 64 bytes at a time, so any write bigger than 64 bytes is truncated and the number of actually written bytes is returned to the caller.

The device has a local buffer of, I think, 64 bytes that contains the textual outputs of the REPL interaction, if this buffer ever fills up the device stops accepting bulk transfers; in the Write function I make sure that the device buffer is always empty after a write by performing successive USB bulk transfer reads until no more bytes are read, the output of these reads is then stored on a local buffer called *read_buffer* that is used when the **Read** function is called.

2.1.6 Read

The Read function does not communicate directly with the physical device, it just copies to userspace the contents of *read_buffer*

My buffer is of fixed size and gets flushed when it overflows or after a full read is completed **i.e.** when the number of read bytes is greater or equal to the number of available bytes.

3 Project outcomes

3.1 Concrete outcomes

The produced artifacts are available in **Github** and are:

- This report
- The project presentation
- The device driver

3.2 Learning outcomes

- This project helped me to understand how to work more effectively in a Linux environment
- I learned how USB endpoint enumeration works and the basics of how the Linux kernel implements communication with USB devices

3.3 Existing knowledge

I think most of the relevant knowledge was given by AOS, the rest of my career did not cover these topics with this level of depth if at all.

A course that helped me better understand C programming in general was Computer Security.

3.4 Problems encountered

The main issue of the project was understanding how my device worked, I could not find any documentation so I had to figure out why it behaved like that by trial and error. This is my first project using a physical device, so this is my first time tackling this type of problems.

4 Honor Pledge

(This part cannot be modified and it is mandatory to sign it)

I/We pledge that this work was fully and wholly completed within the criteria established for academic integrity by Politecnico di Milano (Code of Ethics and Conduct) and represents my/our original production, unless otherwise cited.

I/We also understand that this project, if successfully graded, will fulfill part B requirement of the Advanced Operating System course and that it will be considered valid up until the AOS exam of Sept. 2026.

Samuele Lo Piccolo